Hierarchical Indexing

Python Data Science Handbook — Chapter 3, Section 3.5

The `MultiIndex` — packing extra dimensions of labels into a 1-D `Series` or 2-D `DataFrame`.

1 Why you would ever want a second level of index

So far an index has been a flat list of labels: one label per row. That is enough for genuinely two-dimensional data (rows × columns). But an enormous amount of real data is naturally *three* or more dimensions: sales by *store* and *month*, gene expression by *patient* and *tissue*, sensor readings by *location* and *timestamp*. You want to keep indexing data — looking it up by meaningful labels — but now along more than one axis at once.

Background & Context

The cleanest tool for N-dimensional labeled data is a genuinely N-dimensional container (the `xarray` library exists for exactly this). But in day-to-day work we usually want to stay inside the `Series`/`DataFrame` world, because all the alignment, grouping, and plotting machinery lives there. **Hierarchical indexing** (a.k.a. multi-level indexing) is Pandas's answer: it encodes the extra axes as nested *levels* of a single index, so a 1-D `Series` can stand in for 2-D data, and a 2-D `DataFrame` can stand in for 3-D or 4-D data.

1.1 The naive approach: Python tuples as keys

Suppose we have a population figure for a few US states, but at *two different years*. The label for each value is really a pair `(state, year)`. A first instinct is to use tuples as the index keys:

```
import numpy as np
import pandas as pd

index = [('California', 2010), ('California', 2020),
         ('New York',   2010), ('New York',   2020),
         ('Texas',      2010), ('Texas',      2020)]
populations = [37_253_956, 39_538_223,
               19_378_102, 20_201_249,
               25_145_561, 29_145_505]
pop = pd.Series(populations, index=index)
print(pop)
```

```
(California, 2010)    37253956
(California, 2020)    39538223
(New York, 2010)      19378102
(New York, 2020)      20201249
(Texas, 2010)         25145561
(Texas, 2020)         29145505
dtype: int64
```

This *works*: `pop[('California', 2020)]` returns the right number. But it is clumsy the moment you want a *slice*. The index is, as far as Pandas is concerned, a flat list of opaque tuple objects. There is no notion that the first element is a "state level" and the second a "year level." So a natural question like "give me every state's 2020 figure" has no clean expression:

```
# Select all years 2020 -- ugly: we must loop the tuples by hand
pop[[i for i in pop.index if i[1] == 2020]]
```

```
(California, 2020)    39538223
(New York, 2020)      20201249
(Texas, 2020)         29145505
dtype: int64
```

That list comprehension is exactly the kind of manual bookkeeping Pandas is supposed to eliminate. It is also slow (a Python loop over every label) and it does not generalize: slicing on the *outer* level, partial selection, sorting by one level — all become bespoke code.

Key Idea

A list of tuples *looks* hierarchical but is not. Pandas sees a flat sequence of tuple objects with no level structure, so none of the level-aware operations (partial indexing, cross-section, group-by-level) are available. The fix is to promote those tuples into a real `MultiIndex`.

1.2 The clean solution: `pd.MultiIndex`

A `MultiIndex` is an index object that explicitly stores *multiple levels*, each its own array of labels, plus the integer codes that say which label of each level a given row uses. We can build one straight from our tuples:

```
mi = pd.MultiIndex.from_tuples(index)
pop = pop.reindex(mi)
print(pop)
```

```
California  2010    37253956
            2020    39538223
New York    2010    19378102
            2020    20201249
Texas       2010    25145561
            2020    29145505
dtype: int64
```

Two things changed. Visually, the repeated outer label (`California`, `New York`, ...) is now collapsed — the blank space under `California` means "same as above," signalling the level structure. Functionally, we now get the slice we wanted with trivial syntax:

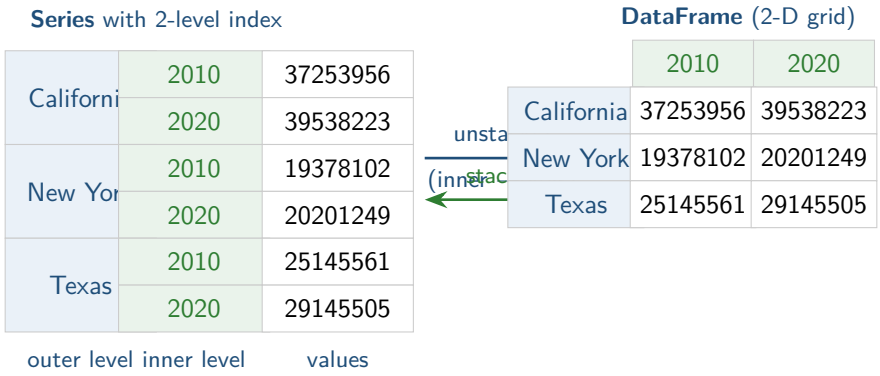
```
pop[:, 2020]      # all rows whose SECOND level == 2020
```

```
California     39538223
New York       20201249
Texas          29145505
dtype: int64
```

The result is a single-level `Series` indexed only by state — Pandas dropped the level we fully specified. This is shorter, faster, and far more readable than the list comprehension.

2 A MultiIndex is literally an extra dimension

Here is the mental model worth internalizing. Our `pop` Series has six entries laid out along one axis, but its *meaning* is a 3×2 grid (three states, two years). The `MultiIndex` is what lets a 1-D container carry 2-D meaning. The conversion between "stacked into a column" and "spread into a grid" is so common it has two named methods: `unstack` and `stack`.




```
pop_df = pop.unstack()    # inner level (year) becomes the columns
pop_df
```

	2010	2020
California	37253956	39538223
New York	19378102	20201249
Texas	25145561	29145505

```
pop_df.stack()    # columns fold back into a second index level
```

And `stack` returns us exactly to the original multiply-indexed `Series`. The pair is a lossless round trip.

Intuition

`unstack` pivots one index level "up" to become columns (data gets *wider*); `stack` pivots columns "down" to become an index level (data gets *taller*). It is the same numbers, reshaped — a `MultiIndex` on rows and a column axis are two views of the same higher-dimensional cube.

2.1 Why bother — just add a column?

Why not keep a plain `DataFrame` with a `year` *column* instead of a year index level? You can, and often should. But once a quantity is a label rather than a value, treating it as an index level unlocks alignment and group-by-level operations, lets you add *more* columns of actual measurements without confusion, and gives you the `stack`/`unstack` reshaping for free. The `MultiIndex` earns its keep precisely when you have several measured columns indexed by the same set of label axes:

```
pop_df = pd.DataFrame({'total': pop,
                       'under18': [9_284_094, 8_898_092,
                                   4_687_374, 4_318_033,
                                   6_879_014, 7_503_437]})
pop_df['frac_under18'] = pop_df['under18'] / pop_df['total']
pop_df['frac_under18'].unstack()
```

```
                2010      2020
California   0.249265  0.225041
New York     0.241886  0.213750
Texas        0.273568  0.257619
dtype: float64
```

The arithmetic ran row-by-row honoring both index levels, then `unstack` reshaped the answer into a readable grid — no manual joining of state and year keys anywhere.

3 Constructing a MultiIndex

You rarely call `from_tuples` by hand. More often a `MultiIndex` appears *implicitly*, or you build it from arrays/products.

3.1 Implicit construction

If you pass a *list of two or more lists* as the `index` to a `Series` or `DataFrame`, Pandas builds the `MultiIndex` for you:

```
df = pd.DataFrame(np.round(np.random.rand(4, 2), 2),
                  index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
                  columns=['data1', 'data2'])
```

Likewise, passing a dictionary whose *keys are tuples* makes Pandas recognize the levels automatically:

```
data = {('California', 2010): 37_253_956,
        ('California', 2020): 39_538_223,
        ('Texas',      2010): 25_145_561,
        ('Texas',      2020): 29_145_505}
pd.Series(data)   # → a MultiIndexed Series, no extra work
```

3.2 Explicit constructors

When you want to be deliberate, the `pd.MultiIndex` class has three workhorse class methods:

```
# From a list of (level0, level1) tuples
pd.MultiIndex.from_tuples([('a', 1), ('a', 2), ('b', 1), ('b', 2)])

# From a list of arrays, one array per level (parallel, row-aligned)
pd.MultiIndex.from_arrays([['a', 'a', 'b', 'b'], [1, 2, 1, 2]])

# From the CARTESIAN PRODUCT of single-level iterables
pd.MultiIndex.from_product([['a', 'b'], [1, 2]])
```

Intuition

Choose by how your levels relate. `from_product` when every combination of level values exists (a full grid — e.g. every state crossed with every year). `from_arrays` when you already have the levels as parallel columns and not all combinations occur. `from_tuples` when the pairs arrive naturally as tuples. They all produce the same kind of object.

3.3 Naming the levels

An unnamed level is hard to refer to later. Every `MultiIndex` has a `names` attribute — a list with one name per level — and you should set it:

```
pop.index.names = ['state', 'year']
print(pop)
```

```
state       year
California  2010    37253956
            2020    39538223
New York    2010    19378102
            2020    20201249
Texas       2010    25145561
            2020    29145505
dtype: int64
```

Now the level names print as a header, and you can use them in `groupby(level='year')`, `xs(level='state')`, and so on, instead of remembering that "year is level 1."

4 Hierarchical indexing on columns too

Rows are not special. A `DataFrame`'s *columns* are themselves an `Index`, so they can be a `MultiIndex` as well. This gives you fully four-dimensional tables: two label axes down the rows, two across the columns. A classic use is repeated measurements — several subjects, each measured for several quantities, across several visits.

```
# hierarchical row index: (year, visit)
rows = pd.MultiIndex.from_product([[2024, 2025], [1, 2]],
                                  names=['year', 'visit'])
# hierarchical column index: (subject, measurement-type)
cols = pd.MultiIndex.from_product([['Ada', 'Bo'], ['hr', 'temp']],
```

```
                                   names=['subject', 'type'])

rng = np.random.default_rng(0)
data = np.round(37 + 3 * rng.standard_normal((4, 4)), 1)
data[:, ::2] = np.round(data[:, ::2] * 2)   # make hr look heart-rate-ish
health = pd.DataFrame(data, index=rows, columns=cols)
health
```

	subject	Ada		Bo	
	type	hr	temp	hr	temp
year	visit				
2024	1	74.0	37.4	78.0	36.6
	2	76.0	38.1	72.0	37.0
2025	1	80.0	36.9	74.0	37.7
	2	70.0	37.2	82.0	36.8

Because the columns are hierarchical, a top-level key selects a whole sub-table — everything for one subject:

```
health['Ada']            # a DataFrame with columns ['hr', 'temp']
health['Ada', 'hr']      # a single Series: Ada's heart rate over time
```

Key Idea

The same partial-indexing rule applies on both axes. A single bare key reaches into the *outermost* level and returns the corresponding lower-dimensional slice. `health['Ada']` is to columns what `pop['California']` is to rows.

5 Indexing and slicing multiply-indexed objects

5.1 On a Series

Take the named `pop` from above. The pattern is: supply labels for levels left-to-right, and Pandas returns whatever is left.

```
pop['California']          # partial index, outer level → a Series over
    year
pop['California', 2010]    # both levels → a single scalar, 37253956
pop['California':'New York']        # outer-level slice (endpoint INCLUSIVE
    )
pop[:, 2020]               # fix the INNER level, keep all outer labels
pop[pop > 22_000_000]      # boolean mask works as usual
pop[['California', 'Texas']]        # fancy indexing on the outer level
```

Common Pitfall

`pop[:, 2020]` — using the empty slice `:` as a placeholder to "keep everything at the outer level while pinning the inner level" — only works because this is a `Series`. On a `DataFrame`, a bare bracket lookup `df[...]` indexes *columns*, so this trick does not transfer; you need `.loc` and (often) `pd.IndexSlice`, covered next.

5.2 On a DataFrame

`DataFrame` selection goes through `.loc` and `.iloc`. With a row `MultiIndex`, the `.loc` key is itself a tuple of per-level labels:

```
health.loc[2024]                # partial: all visits in 2024 (a DataFrame)
health.loc[(2024, 1)]           # both row levels → one row (a Series)
health.loc[(2024, 1), ('Ada', 'hr')]   # one cell, addressing all 4 levels
```

The trouble starts when you want to slice the *inner* level. You might reach for `health.loc[(:, 1), :]` to mean "visit 1 in every year," but a bare colon is illegal *inside* a tuple — Python parses `:` as slice syntax only in subscripts, not inside parentheses. That syntax error is what `pd.IndexSlice` exists to dodge.

5.3 `pd.IndexSlice` for inner-level slicing

`pd.IndexSlice` is a tiny helper object whose only job is to let you write slice syntax (`:`, `start:stop`) *inside* a `.loc` key, per level:

```
idx = pd.IndexSlice

# every year, but only visit 1; and only the 'hr' column of each subject
health.loc[idx[:, 1], idx[:, 'hr']]
```

```
subject          Ada    Bo
                  hr    hr
year visit
2024 1          74.0  78.0
2025 1          80.0  74.0
```

Read `idx[:, 1]` as "all of level 0 (year), level 1 (visit) equal to 1," and `idx[:, 'hr']` as the analogous slice across the column levels. `IndexSlice` is purely syntactic sugar — it builds the right tuple of slice objects so you can express row *and* column inner-level selection in a single `.loc` call.

6 Rearranging multi-indexed data

6.1 Sorting: why slicing demands a sorted index

A `MultiIndex` stores its labels as *codes* pointing into per-level label arrays. Range slicing on those levels (`'California':'New York'`, or any partial outer slice) is only well-defined and only *fast* when the index is **lexically sorted** — because a slice is internally a search for a contiguous block of codes,

and "contiguous" only means anything if the labels are in order. If you try to slice an unsorted `MultiIndex`, Pandas refuses:

```
bad = pd.Series(range(4),
                index=pd.MultiIndex.from_tuples(
                    [('b', 1), ('a', 1), ('b', 2), ('a', 2)]))
bad['a':'b']
# UnsortedIndexError: 'Key length (1) was greater than MultiIndex
#   lexsort depth (0)'
```

The cure is one call:

```
good = bad.sort_index()
good['a':'b']          # now works -- the block ('a',1),('a',2) is
    contiguous
```

Common Pitfall

`UnsortedIndexError` (or the older `KeyError` about lexsort depth) almost always means "you sliced a `MultiIndex` you never sorted." The fix is essentially always `df = df.sort_index()` right after you build or load the data. Sorting once up front makes every later partial slice both legal and fast.

6.2 `reset_index` and `set_index`

Sometimes a flat table with the levels as ordinary columns is more convenient (for merging, plotting, or writing to CSV). `reset_index` turns index levels into columns; `set_index` does the reverse.

```
flat = pop.reset_index(name='population')
#   state        year   population
# 0 California   2010    37253956
# 1 California   2020    39538223
# ...
flat.set_index(['state', 'year'])   # back to a MultiIndexed object
```

Intuition

`reset_index` ↔ `set_index` is the "index ↔ column" round trip, just as `stack` ↔ `unstack` is the "rows ↔ columns" round trip. Between these two pairs you can move any label axis to wherever it is most convenient for the operation at hand.

6.3 `stack`/`unstack` with a chosen level

When more than two levels are in play, tell `unstack` *which* level to pivot using `level=` (by position or by name):

```
pop.unstack(level=0)    # 'state' becomes the columns
pop.unstack(level=1)    # 'year' becomes the columns (the default:
    innermost)
```

```
pop.unstack(level='year')   # same, by name -- clearer
```

7 Aggregating across a level

The real payoff of named levels is group-wise reduction. To collapse one axis — say, average over years to get a per-state mean — group by the levels you want to *keep* and reduce:

```
pop.groupby(level='state').mean()
```

```
state
California    38396089.5
New York      19789675.5
Texas         27145533.0
dtype: float64
```

For a `DataFrame` with hierarchical columns, `groupby` also accepts an `axis` and a column-level name, so you can aggregate across, say, subjects while keeping measurement type:

```
# mean over both subjects, per (year, visit) row and per measurement type
health.groupby(level='type', axis=1).mean()
```

Common Pitfall

You may see older code written as `pop.mean(level='state')` — passing `level=` straight to a reduction like `mean`, `sum`, or `std`. That form is **deprecated** (removed in pandas 2.0). Always spell it `pop.groupby(level='state').mean()` instead. It is more explicit about what is happening (group, then reduce) and it is the form that will keep working.

Key Idea

`groupby(level=...)` is just `groupby` where the grouping keys come from *index levels* rather than columns. Everything you know about `groupby` — multiple aggregations via `.agg`, custom functions, transform — applies unchanged. The `MultiIndex` simply hands `groupby` a ready-made set of keys.

Section recap

- A `MultiIndex` packs N-dimensional labeled data into a 1-D `Series` or 2-D `DataFrame` by giving the index multiple *levels*. Tuples-as-keys look similar but lack level structure — promote them with `pd.MultiIndex`.
- Build one implicitly (list-of-lists index, or dict with tuple keys) or explicitly via `from_tuples`, `from_arrays`, `from_product`; always set `index.names`.
- A row `MultiIndex` and the column axis are two views of one cube: `unstack` widens (level → columns), `stack` narrows (columns → level). Columns can be hierarchical too.
- Index left-to-right by level; a bare outer key returns a lower-dimensional slice. Use `.loc` with

`pd.IndexSlice` to slice *inner* levels on a `DataFrame`.

- `sort_index()` before slicing a `MultiIndex`, or you get `UnsortedIndexError`. `reset_index` / `set_index` move levels to and from columns.
- Aggregate across a level with `groupby(level=...)`; the older `.mean(level=...)` form is deprecated.